

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG**

**HUỲNH PHẠM LONG TRIỀU - 23521655
NGUYỄN VĂN TRƯỜNG - 23521693**

ĐỒ ÁN CHUYÊN NGÀNH

**PHƯƠNG PHÁP KIỂM THỬ HỘP ĐEN
TỰ ĐỘNG CHO CÁC API WEB
RESTFUL**

**AUTOMATED BLACK-BOX TESTING
FOR RESTFUL WEB API**

KỸ SƯ NGÀNH AN TOÀN THÔNG TIN

TP. HỒ CHÍ MINH, NĂM 2026

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG**

**HUỲNH PHẠM LONG TRIỀU - 23521655
NGUYỄN VĂN TRƯỜNG - 23521693**

ĐỒ ÁN CHUYÊN NGÀNH

**PHƯƠNG PHÁP KIỂM THỬ HỘP ĐEN
TỰ ĐỘNG CHO CÁC API WEB
RESTFUL**

**AUTOMATED BLACK-BOX TESTING
FOR RESTFUL WEB API**

KỸ SƯ NGÀNH AN TOÀN THÔNG TIN

**GIẢNG VIÊN HƯỚNG DẪN
ThS. NGÔ KHÁNH KHOA**

TP. HỒ CHÍ MINH, NĂM 2026

Lời cảm ơn

Không ai đạt được điều gì đó to lớn mà không nhờ sự giúp đỡ của những người xung quanh, cho dù là trực tiếp hay gián tiếp đi nữa. Để hoàn thành được khóa luận này, tác giả may mắn nhận được nhiều sự giúp đỡ và hỗ trợ từ quý thầy, cô, anh chị, bạn bè và người thân. Tác giả xin dành những trang đầu tiên này để bày tỏ lòng tri ân của mình tới tất cả mọi người, những người đã đồng hành cùng nhóm trong khoảng thời gian vừa qua.

Đầu tiên, tác giả xin gửi lời cảm ơn sâu sắc đến toàn thể các thầy cô của Trường Đại học Công nghệ Thông tin nói chung và các thầy cô khoa Mạng máy tính và Truyền thông nói riêng. Nhờ những kiến thức quý giá mà thầy cô đã truyền đạt, cũng như việc hỗ trợ tận tình trong suốt khoảng thời gian thực hiện, nhóm đã hoàn thành khóa luận và đạt được các kết quả đáng ghi nhận.

Tác giả xin đặc biệt cảm ơn ThS. Ngô Khánh Khoa là người đã truyền cảm hứng, tận tình hướng dẫn và hỗ trợ tận tình về kiến thức, tạo môi trường thuận lợi để nhóm có thể học hỏi, trao đổi với các bạn, các em trong nhóm nghiên cứu. Đây là những kiến thức, kinh nghiệm quý giá, không chỉ có tác dụng trong khóa luận tốt nghiệp này mà còn trong khoảng thời gian làm việc trong chặng đường tiếp theo.

Cuối cùng, tôi xin bày tỏ lòng tri ân đến gia đình và người thân, những người đã luôn là những hậu phương vững chắc và luôn ủng hộ từng quyết định mà nhóm đưa ra.

Mặc dù đã nỗ lực rất nhiều để luận văn được hoàn thiện nhất, song khó có thể tránh khỏi thiếu sót và hạn chế. Kính mong nhận được sự thông cảm và ý kiến đóng góp từ quý thầy cô và các bạn.

TP. Hồ Chí Minh, Ngày 22 tháng 6 năm 2026

Nguyễn Văn Trường

Mục lục

Lời cảm ơn	i
Mục lục	ii
Danh sách hình vẽ	vi
Danh sách bảng	vii
Danh sách đoạn mã	viii
Tóm tắt đồ án	1
1 Giới thiệu	3
1.1 Bối cảnh	3
1.2 Phát biểu bài toán	4
1.3 Mục tiêu và nhiệm vụ	5
1.3.1 Mục tiêu tổng quát	5
1.3.2 Nhiệm vụ cụ thể	5
1.4 Hướng tiếp cận	5
1.4.1 Phân tích tĩnh dựa trên đặc tả API	5
1.4.2 Thực thi kiểm thử có duy trì trạng thái	6
1.4.3 Thực thi bất đồng bộ	6
1.5 Đóng góp của đồ án	6
1.5.1 Về mặt kiến trúc	6
1.5.2 Về mặt kỹ thuật	6
1.5.3 Về mặt ứng dụng	6

1.5.4	Về hướng phát triển	7
2	TỔNG QUAN TÀI LIỆU VÀ CÁC NGHIÊN CỨU LIÊN QUAN	8
2.1	Các công trình liên quan trong lĩnh vực	8
2.1.1	Nhóm công cụ Fuzzing không trạng thái (Stateless/Dumb Fuzzing)	8
2.1.2	Nhóm công cụ Fuzzing duy trì trạng thái dựa trên ngữ pháp (Stateful & Grammar-based Fuzzing)	9
2.1.3	Nhóm tiếp cận dựa trên đồ thị và thuật toán tìm kiếm (Graph-based & Heuristic Search)	10
2.2	Phân tích và khoảng trống nghiên cứu	10
2.2.1	Độ phức tạp khi xây dựng đồ thị phụ thuộc API	11
2.2.2	Hạn chế về hiệu năng thực thi	11
2.2.3	Vấn đề quản lý trạng thái	11
2.2.4	Khả năng tích hợp trí tuệ nhân tạo	11
2.3	Định hướng của đề án	12
3	PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ HỆ THỐNG	14
3.1	Kiến trúc tổng thể của hệ thống	14
3.1.1	Module phân tích đặc tả (Spec Parser)	15
3.1.2	Module xây dựng đồ thị phụ thuộc API (Operation Dependency Graph)	15
3.1.3	Module quản lý trạng thái và thực thi (Runtime Executor State-Store)	17
3.1.4	Module đột biến bất đồng bộ (Async Mutator)	17
3.1.5	Module suy luận ngữ nghĩa (Rule Inference Layer)	18
3.1.6	Module sinh payload thông minh (LLM Planner)	18
3.1.7	Module bộ máy chiến lược kiểm thử (Test Strategy Engine) . .	19
3.1.8	Module thực thi HTTP (Runtime Executor)	19
3.1.9	Module bộ nhớ thời gian chạy (Knowledge Memory)	19
3.2	Phân tích tĩnh và xây dựng đồ thị phụ thuộc API	20
3.2.1	Bài toán xây dựng quan hệ phụ thuộc	20
3.2.2	Inverted Semantic Index	20
3.2.3	Cơ chế tự phục hồi payload (Self-Healing)	21
3.2.4	Mô hình tính điểm phụ thuộc	21

3.2.5	Độ tương đồng Jaccard	21
3.2.6	IDF Penalty Weight	22
3.2.7	Method Directionality Score	22
3.3	Quản lý trạng thái Stateful Execution	22
3.3.1	Trích xuất trạng thái động	22
3.3.2	Cơ chế Cloneable StateStore	23
3.4	Phương pháp đột biến và bắn phá bất đồng bộ	23
3.4.1	Fuzz Dictionary	23
3.4.2	Async Payload Blasting	23
3.5	Runtime Feedback và hệ thống báo cáo	24
3.6	Kết luận chương	25
4	TRIỂN KHAI HỆ THỐNG VÀ THỰC NGHIỆM	26
4.1	Môi trường triển khai	26
4.2	Cấu trúc chương trình	28
4.3	Hiện thực các module chính	29
4.3.1	Module xây dựng đồ thị phụ thuộc API	29
4.3.2	Module quản lý trạng thái StateStore	30
4.3.3	Module Local Mutator và Async Fuzz Engine	30
4.4	Hiện thực các thuật toán toán học	32
4.4.1	Tính Jaccard Similarity	32
4.4.2	Tính IDF Penalty Weight	32
4.4.3	Tính Confidence Score	32
4.5	Quy trình thực thi hệ thống	33
4.6	Kịch bản thực nghiệm	33
4.6.1	Giai đoạn xây dựng ngữ cảnh	33
4.6.2	Giai đoạn Fuzzing	34
4.6.3	Giai đoạn ghi nhận kết quả	34
4.7	Cơ chế tự phục hồi payload (Self-Healing)	34
4.8	Kết luận chương	35
5	THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	36
5.0.1	Thiết lập môi trường và Tiêu chuẩn đánh giá	36
5.0.2	Báo cáo kết quả định lượng tổng hợp	37

6	ĐỀ CƯƠNG KHÓA LUẬN TỐT NGHIỆP	38
6.1	Đặt vấn đề và Tổng quan	38
6.2	Mục tiêu và Nhiệm vụ nghiên cứu	39
6.2.1	Mục tiêu	39
6.2.2	Nhiệm vụ cụ thể	39
6.3	Phương pháp luận và Nội dung dự định thực hiện	40
6.3.1	Phát triển phương pháp lập kế hoạch tấn công tự động bằng Multi-Agent LLM	40
6.3.2	Tối ưu hóa quá trình tìm kiếm chuỗi tấn công	41
6.4	Đánh giá kết quả	42
6.4.1	Dữ liệu thực nghiệm	42
6.4.2	Mô hình đối chứng	42
6.4.3	Các tiêu chí đánh giá	42
6.4.4	Tiêu chí thành công	43
A	Hướng dẫn soạn phụ lục	44
	Tài liệu tham khảo	45

Danh sách hình vẽ

Danh sách bảng

5.1	Mức độ bao phủ và mật độ lỗi hỏng phần mềm phía sau cho mỗi công cụ .	37
-----	---	----

Danh sách đoạn mã

Tóm tắt đồ án

Sự phát triển mạnh mẽ của kiến trúc vi dịch vụ (Microservices) và các ứng dụng nền tảng web đã khiến RESTful API trở thành một thành phần quan trọng trong các hệ thống phần mềm hiện đại. Tuy nhiên, sự mở rộng nhanh chóng của các hệ thống API cũng đồng thời làm gia tăng bề mặt tấn công, tạo ra nhiều nguy cơ về bảo mật. Các phương pháp kiểm thử bảo mật API truyền thống, đặc biệt là kiểm thử thủ công hoặc các kỹ thuật Fuzzing không có nhận thức về ngữ cảnh (Dumb Fuzzing), thường gặp hạn chế trong việc phát hiện các lỗ hổng liên quan đến logic nghiệp vụ phức tạp.

Nhằm giải quyết vấn đề trên, đồ án đề xuất xây dựng một phương pháp kiểm thử hộp đen tự động dựa trên kiến trúc Hybrid Stateful API Fuzzer. Thay vì thực hiện các yêu cầu kiểm thử một cách ngẫu nhiên và độc lập, hệ thống khai thác tài liệu đặc tả OpenAPI để phân tích cấu trúc API, trích xuất thông tin ngữ nghĩa và xây dựng Đồ thị phụ thuộc hoạt động (Operation Dependency Graph - ODG). Đồ thị này được tạo dựng dựa trên thuật toán Inverted Semantic Index nhằm xác định mối quan hệ giữa các API và tối ưu hóa quá trình sinh chuỗi kiểm thử.

Dựa trên mô hình ODG, hệ thống áp dụng cơ chế Stateful Fuzzing nhằm duy trì trạng thái trong quá trình kiểm thử. Bộ máy thực thi (Runtime Executor) có khả năng tự động thu thập, lưu trữ và tái sử dụng các tham số phụ thuộc giữa các API, chẳng hạn như token xác thực hoặc định danh tài nguyên, từ đó xây dựng các chuỗi tương tác API hợp lệ để kiểm tra các trường hợp lỗi tiềm ẩn. Bên cạnh đó, cơ chế đột biến dữ liệu bất đồng bộ (Asynchronous Payload Blasting) được tích hợp nhằm tăng khả năng kiểm thử với số lượng lớn dữ liệu bất thường, hỗ trợ phát hiện các lỗi xử lý đầu vào, lỗi logic nghiệp vụ và các sự cố bất thường từ máy chủ.

Kết quả của đồ án là một hệ thống tự động hóa kiểm thử bảo mật RESTful API theo hướng hộp đen, có khả năng phát hiện các lỗ hổng mà không yêu cầu quyền truy cập vào mã nguồn ứng dụng. Hệ thống đồng thời hỗ trợ sinh báo cáo trực quan dưới dạng

Dashboard HTML, cung cấp thông tin chi tiết về lịch sử request/response nhằm phục vụ quá trình phân tích và đánh giá. Đồ án tạo nền tảng kiến trúc và dữ liệu cần thiết cho các hướng nghiên cứu mở rộng trong tương lai, đặc biệt là việc tích hợp các mô hình trí tuệ nhân tạo (Multi-Agent LLM) nhằm nâng cao khả năng phân tích và tự động hóa quá trình kiểm thử bảo mật API.

Chương 1

Giới thiệu

1.1 Bối cảnh

Trong xu hướng chuyển đổi số hiện nay, các hệ thống phần mềm đang dần chuyển dịch từ kiến trúc nguyên khối (Monolithic) sang các mô hình phân tán như vi dịch vụ (Microservices), ứng dụng trang đơn (Single Page Application - SPA) và ứng dụng di động. Trong các kiến trúc này, API (Application Programming Interface), đặc biệt là RESTful API, đóng vai trò quan trọng trong việc trao đổi dữ liệu và kết nối giữa các thành phần của hệ thống.

Tuy nhiên, sự phụ thuộc ngày càng lớn vào API cũng làm gia tăng bề mặt tấn công (Attack Surface) của hệ thống. Khác với các ứng dụng web truyền thống, nơi các rủi ro thường tập trung ở tầng giao diện người dùng như Cross-Site Scripting (XSS) hoặc Clickjacking, các vấn đề bảo mật của API thường liên quan đến dữ liệu, cơ chế xác thực và logic xử lý nghiệp vụ.

Theo các báo cáo về bảo mật API, nhiều lỗ hổng nghiêm trọng thường xuất hiện trong quá trình thiết kế và triển khai API như Broken Object Level Authorization (BOLA), Broken Function Level Authorization (BFLA), lỗi kiểm tra dữ liệu đầu vào hoặc các lỗi xử lý ngoại lệ dẫn đến sự cố hệ thống. Vì vậy, việc xây dựng các phương pháp kiểm thử bảo mật API tự động, có khả năng phát hiện các vấn đề tiềm ẩn trong giai đoạn phát triển phần mềm là một yêu cầu quan trọng đối với các hệ thống hiện đại.

1.2 Phát biểu bài toán

Hiện nay, việc kiểm thử bảo mật API chủ yếu được thực hiện thông qua hai phương pháp chính: kiểm thử thủ công (Manual Penetration Testing) và các công cụ kiểm thử tự động (Automated Scanner/Fuzzer). Tuy nhiên, cả hai phương pháp vẫn tồn tại nhiều hạn chế.

Đối với kiểm thử thủ công, quá trình phân tích và xây dựng các kịch bản tấn công trên những hệ thống có số lượng lớn API Endpoint yêu cầu nhiều thời gian và nguồn lực. Ngoài ra, khả năng bao phủ (Coverage) toàn bộ các trường hợp kiểm thử còn phụ thuộc nhiều vào kinh nghiệm của chuyên gia.

Đối với các kỹ thuật Fuzzing truyền thống (Stateless Fuzzing), các công cụ thường xem mỗi API Endpoint là một thành phần độc lập và gửi các dữ liệu kiểm thử một cách riêng lẻ. Phương pháp này không duy trì trạng thái (State) của phiên làm việc, dẫn đến khó khăn trong việc kiểm thử các API có quan hệ phụ thuộc lẫn nhau.

Ví dụ, để kiểm thử một API cập nhật thông tin người dùng, hệ thống cần thực hiện chuỗi thao tác bao gồm đăng nhập để lấy Token xác thực, tạo hoặc truy xuất thông tin người dùng để lấy định danh tài nguyên, sau đó sử dụng các dữ liệu này trong yêu cầu cập nhật. Các Fuzzer không có khả năng quản lý trạng thái thường không thể vượt qua các bước xác thực ban đầu, do đó bỏ sót các lỗi logic nghiệp vụ nằm sâu trong chuỗi tương tác API.

Vì vậy, bài toán đặt ra là xây dựng một hệ thống kiểm thử hộp đen tự động có khả năng:

- Phân tích mối quan hệ phụ thuộc giữa các API.
- Duy trì trạng thái trong suốt quá trình thực thi chuỗi kiểm thử.
- Tự động sinh các kịch bản kiểm thử phù hợp.
- Thực thi số lượng lớn yêu cầu nhằm phát hiện các lỗi xử lý bất thường của hệ thống.

1.3 Mục tiêu và nhiệm vụ

1.3.1 Mục tiêu tổng quát

Xây dựng kiến trúc cốt lõi của hệ thống Hybrid Stateful API Fuzzer, một công cụ kiểm thử hộp đen tự động dành cho RESTful API dựa trên tài liệu đặc tả OpenAPI. Hệ thống tập trung vào khả năng phân tích quan hệ phụ thuộc giữa các API, duy trì trạng thái thực thi và phát hiện các lỗi bất thường trong quá trình xử lý.

1.3.2 Nhiệm vụ cụ thể

Các nhiệm vụ chính của đề án bao gồm:

- Nghiên cứu các kiến thức nền tảng về RESTful API, OpenAPI Specification và các vấn đề bảo mật API phổ biến.
- Xây dựng module phân tích tĩnh (Static Analysis) nhằm đọc tài liệu OpenAPI và trích xuất đồ thị phụ thuộc hoạt động (Operation Dependency Graph - ODG).
- Phát triển cơ chế quản lý trạng thái (StateStore) để lưu trữ và luân chuyển các dữ liệu phụ thuộc như Token xác thực, ID tài nguyên giữa các API.
- Xây dựng module sinh và thực thi dữ liệu kiểm thử bất đồng bộ (Asynchronous Payload Blasting) nhằm nâng cao khả năng phát hiện lỗi.
- Thiết kế hệ thống báo cáo trực quan (Dashboard) giúp theo dõi và phân tích kết quả kiểm thử.

1.4 Hướng tiếp cận

Để giải quyết bài toán trên, đề án áp dụng hướng tiếp cận kiểm thử kết hợp duy trì trạng thái (Hybrid Stateful Testing), bao gồm các thành phần chính:

1.4.1 Phân tích tĩnh dựa trên đặc tả API

Hệ thống sử dụng tài liệu OpenAPI làm nguồn dữ liệu đầu vào để phân tích cấu trúc API. Thông qua việc trích xuất thông tin về Endpoint, tham số và mô hình dữ liệu, hệ thống xây dựng đồ thị phụ thuộc giữa các hoạt động API.

1.4.2 Thực thi kiểm thử có duy trì trạng thái

Thay vì kiểm thử từng API độc lập, hệ thống duy trì trạng thái thực thi thông qua StateStore. Các dữ liệu sinh ra trong quá trình thực hiện một API có thể được tái sử dụng cho các API tiếp theo, từ đó tạo ra các chuỗi kiểm thử gắn với hành vi người dùng thực tế.

1.4.3 Thực thi bất đồng bộ

Hệ thống áp dụng cơ chế xử lý bất đồng bộ (Asynchronous Execution) nhằm tăng hiệu suất gửi yêu cầu kiểm thử. Phương pháp này cho phép thực hiện nhiều trường hợp kiểm thử đồng thời, hỗ trợ phát hiện các lỗi xử lý dữ liệu hoặc lỗi ngoại lệ từ phía máy chủ.

1.5 Đóng góp của đề án

Đề án mang lại các đóng góp chính như sau:

1.5.1 Về mặt kiến trúc

Đề xuất mô hình Hybrid Stateful API Fuzzer với sự phân tách giữa lớp phân tích tĩnh (Static Analysis) và lớp thực thi động (Runtime Execution). Kiến trúc này giúp khắc phục hạn chế của các phương pháp Fuzzing truyền thống trong việc xử lý các API có quan hệ phụ thuộc.

1.5.2 Về mặt kỹ thuật

Xây dựng cơ chế phân tích và tạo đồ thị phụ thuộc API, kết hợp với bộ quản lý trạng thái nhằm hỗ trợ quá trình sinh chuỗi kiểm thử có ngữ cảnh.

1.5.3 Về mặt ứng dụng

Phát triển công cụ kiểm thử bảo mật API tự động, hỗ trợ phát hiện các lỗi xử lý bất thường và cung cấp báo cáo trực quan phục vụ quá trình đánh giá.

1.5.4 Về hướng phát triển

Kiến trúc hệ thống được xây dựng tạo nền tảng cho việc mở rộng các phương pháp kiểm thử thông minh hơn trong tương lai, đặc biệt là khả năng tích hợp các mô hình trí tuệ nhân tạo nhằm hỗ trợ phân tích và tự động hóa quá trình kiểm thử bảo mật API.

Chương 2

TỔNG QUAN TÀI LIỆU VÀ CÁC NGHIÊN CỨU LIÊN QUAN

2.1 Các công trình liên quan trong lĩnh vực

Các phương pháp kiểm thử hộp đen (Black-box Testing) tự động cho Web API đã trải qua nhiều giai đoạn phát triển. Dựa trên mức độ nhận thức về ngữ cảnh (Context Awareness), các công trình và công cụ hiện nay có thể được chia thành ba nhóm chính.

2.1.1 Nhóm công cụ Fuzzing không trạng thái (Stateless/Dumb Fuzzing)

Đây là nhóm phương pháp kiểm thử tự động phổ biến, trong đó các công cụ dựa trên tài liệu đặc tả API như OpenAPI Specification để sinh các yêu cầu kiểm thử. Các dữ liệu đầu vào thường được tạo tự động dựa trên kiểu dữ liệu được mô tả trong schema hoặc được thay thế bằng các giá trị bất thường nhằm đánh giá khả năng xử lý lỗi của hệ thống.

Dredd và Schemathesis

Dredd là một công cụ kiểm thử API dựa trên đặc tả, tập trung vào việc kiểm tra tính tương thích giữa tài liệu mô tả API và triển khai thực tế. Công cụ sử dụng API description

2.1. Các công trình liên quan trong lĩnh vực

để tự động sinh các request kiểm thử và đánh giá kết quả phản hồi của hệ thống [1].

Bên cạnh đó, Schemathesis áp dụng phương pháp property-based testing để sinh dữ liệu kiểm thử dựa trên OpenAPI hoặc GraphQL schema. Phương pháp này cho phép tự động tạo nhiều trường hợp đầu vào khác nhau nhằm phát hiện các lỗi xử lý dữ liệu [6].

Tuy nhiên, các công cụ thuộc nhóm này chủ yếu xử lý từng API độc lập và không duy trì trạng thái giữa các lần gọi. Vì vậy, khả năng phát hiện các lỗi liên quan đến chuỗi nghiệp vụ nhiều bước còn hạn chế.

OWASP ZAP và Burp Suite

OWASP ZAP là một công cụ kiểm thử bảo mật ứng dụng web mã nguồn mở, cung cấp khả năng quét tự động nhằm phát hiện các lỗi phổ biến như Injection, lỗi cấu hình bảo mật và các vấn đề liên quan đến xử lý dữ liệu đầu vào [4].

Burp Suite là một nền tảng kiểm thử bảo mật được sử dụng rộng rãi trong lĩnh vực kiểm thử xâm nhập. Công cụ cung cấp các chức năng phân tích request/response, proxy, scanner và hỗ trợ kiểm tra nhiều loại lỗ hổng ứng dụng web [5].

Mặc dù có khả năng phát hiện nhiều lỗi ở mức tham số dữ liệu, các công cụ này thường tập trung vào việc kiểm tra từng request riêng lẻ, do đó gặp khó khăn khi cần phân tích các luồng nghiệp vụ phức tạp.

2.1.2 Nhóm công cụ Fuzzing duy trì trạng thái dựa trên ngữ pháp (Stateful & Grammar-based Fuzzing)

Nhằm khắc phục hạn chế của Stateless Fuzzing, các nghiên cứu gần đây tập trung vào khả năng duy trì trạng thái và xây dựng chuỗi gọi API có quan hệ phụ thuộc.

RESTler

RESTler là một trong những nghiên cứu tiên phong về Stateful REST API Fuzzing được giới thiệu tại hội nghị ICSE 2019 [3].

Khác với các công cụ Fuzzing truyền thống, RESTler phân tích tài liệu đặc tả API để suy luận mối quan hệ producer-consumer giữa các API. Ví dụ, một API tạo tài nguyên có thể sinh ra một giá trị định danh (ID), sau đó giá trị này được sử dụng làm đầu vào cho các API truy xuất hoặc cập nhật tài nguyên.

2.2. Phân tích và khoảng trống nghiên cứu

Dựa trên các quan hệ phụ thuộc này, RESTler có khả năng tự động tạo ra các chuỗi request hợp lệ và sử dụng phản hồi từ hệ thống để cải thiện quá trình sinh test case [3].

Tuy nhiên, việc xây dựng quan hệ phụ thuộc giữa các API trong RESTler vẫn có thể gặp giới hạn khi xử lý các hệ thống lớn với số lượng endpoint và tham số phức tạp.

EvoMaster

EvoMaster là một công cụ sinh test case tự động cho RESTful API dựa trên các thuật toán tiến hóa (Evolutionary Algorithms). Công cụ hướng tới việc tối ưu hóa quá trình tìm kiếm nhằm tăng độ bao phủ kiểm thử và phát hiện lỗi trong ứng dụng [2].

Khác với RESTler tập trung vào chuỗi thực thi API, EvoMaster sử dụng cơ chế tìm kiếm dựa trên độ phù hợp (Fitness Function) để cải thiện khả năng khám phá các trường hợp kiểm thử phức tạp [2].

Tuy nhiên, phương pháp này chủ yếu hướng tới kiểm thử dựa trên mã nguồn hoặc môi trường có khả năng quan sát bên trong hệ thống, do đó chưa hoàn toàn phù hợp với bài toán kiểm thử hộp đen API độc lập.

2.1.3 Nhóm tiếp cận dựa trên đồ thị và thuật toán tìm kiếm (Graph-based & Heuristic Search)

Một số nghiên cứu gần đây sử dụng mô hình đồ thị để biểu diễn mối quan hệ giữa các API Endpoint. Trong đó, Đồ thị phụ thuộc hoạt động (Operation Dependency Graph - ODG) được sử dụng để mô hình hóa sự phụ thuộc dữ liệu giữa các hoạt động API.

Sau khi xây dựng đồ thị, các thuật toán tìm kiếm như Breadth-First Search (BFS), Random Walk hoặc Monte Carlo Tree Search (MCTS) có thể được áp dụng nhằm khám phá các luồng nghiệp vụ tiềm năng.

Cách tiếp cận dựa trên đồ thị giúp mở rộng khả năng phát hiện các lỗi logic mà các phương pháp Fuzzing truyền thống khó tiếp cận.

2.2 Phân tích và khoảng trống nghiên cứu

Mặc dù các công trình hiện tại đã đạt được nhiều kết quả quan trọng trong lĩnh vực kiểm thử API tự động, vẫn tồn tại một số hạn chế.

2.2.1 Độ phức tạp khi xây dựng đồ thị phụ thuộc API

Một vấn đề quan trọng là chi phí xây dựng đồ thị phụ thuộc giữa các API. Một số phương pháp cần thực hiện việc so sánh trực tiếp giữa các tham số đầu vào và đầu ra nhằm tìm kiếm mối quan hệ dữ liệu.

Với hệ thống có số lượng lớn API Endpoint, việc đối chiếu N tham số với M tham số có thể dẫn đến độ phức tạp $O(N \times M)$, gây ảnh hưởng đến hiệu năng trong giai đoạn phân tích tĩnh.

2.2.2 Hạn chế về hiệu năng thực thi

Các hệ thống Stateful Fuzzing hiện nay thường ưu tiên tính chính xác của chuỗi thực thi hơn tốc độ gửi request. Do đó, nhiều công cụ vẫn thực hiện quá trình kiểm thử theo hướng tuần tự.

Điều này gây hạn chế khi cần gửi số lượng lớn payload nhằm phát hiện các lỗi xử lý bất thường hoặc lỗi gây mất ổn định hệ thống.

2.2.3 Vấn đề quản lý trạng thái

Khi số lượng chuỗi kiểm thử tăng lên, việc quản lý trạng thái trở nên phức tạp. Việc chia sẻ trạng thái giữa nhiều luồng thực thi có thể gây ra sự ảnh hưởng chéo dữ liệu và tạo ra các kết quả sai lệch.

Do đó, cần có một cơ chế quản lý trạng thái có khả năng sao chép và cô lập dữ liệu giữa các nhánh thực thi.

2.2.4 Khả năng tích hợp trí tuệ nhân tạo

Các hệ thống Fuzzing hiện nay chủ yếu dựa trên các luật hoặc heuristic được định nghĩa trước. Để tích hợp các mô hình trí tuệ nhân tạo trong tương lai, hệ thống cần chuẩn hóa dữ liệu thực thi như trạng thái, lịch sử request/response và các dấu vết thực thi dưới dạng dữ liệu có cấu trúc.

2.3 Định hướng của đề án

Từ các khoảng trống nghiên cứu trên, đề án đề xuất kiến trúc Hybrid Stateful API Fuzzer với các hướng tiếp cận chính:

- Áp dụng Inverted Semantic Index nhằm tối ưu quá trình xác định quan hệ phụ thuộc giữa các API, giảm chi phí xây dựng Operation Dependency Graph.
- Tích hợp cơ chế thực thi bất đồng bộ (Asynchronous I/O) nhằm tăng tốc độ gửi payload và nâng cao khả năng phát hiện lỗi.
- Xây dựng mô hình Cloneable StateStore nhằm quản lý trạng thái độc lập giữa các chuỗi thực thi.
- Chuẩn hóa dữ liệu trạng thái và Execution Trace nhằm tạo nền tảng cho khả năng tích hợp các mô hình AI/Multi-Agent trong các nghiên cứu tiếp theo.

graphicx float

Chương 3

PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ HỆ THỐNG

Trong chương này, đồ án trình bày phương pháp luận và thiết kế hệ thống Hybrid Stateful API Fuzzer được xây dựng nhằm giải quyết các hạn chế của các phương pháp kiểm thử API tự động truyền thống.

Khác với các kỹ thuật Fuzzing không trạng thái (Stateless Fuzzing), hệ thống được thiết kế dựa trên ba thành phần chính: phân tích tĩnh tài liệu đặc tả OpenAPI để xây dựng nhận thức về cấu trúc API, quản lý trạng thái trong quá trình thực thi nhằm tạo các chuỗi gọi API hợp lệ, và cơ chế đột biến dữ liệu bất đồng bộ nhằm tăng khả năng phát hiện các lỗi xử lý bất thường.

Mục tiêu của phương pháp là xây dựng một quy trình kiểm thử hộp đen tự động có khả năng tự động suy luận quan hệ phụ thuộc giữa các API, duy trì ngữ cảnh thực thi và phát hiện các lỗi như Unhandled Exception, lỗi xử lý dữ liệu đầu vào hoặc lỗi gây ra phản hồi HTTP 500.

3.1 Kiến trúc tổng thể của hệ thống

Hệ thống Hybrid Stateful API Fuzzer được thiết kế theo mô hình Data Pipeline gồm bốn phân hệ chính. Mỗi phân hệ đảm nhận một giai đoạn riêng trong quá trình phân tích và kiểm thử API.

3.1.1 Module phân tích đặc tả (Spec Parser)

Spec Parser là thành phần đầu tiên trong hệ thống, chịu trách nhiệm xử lý tài liệu đặc tả OpenAPI.

Module này thực hiện các chức năng:

- Đọc và phân tích tài liệu OpenAPI/Swagger.
- Trích xuất danh sách Endpoint.
- Thu thập HTTP Method của từng API.
- Phân tích Request Parameter và Response Schema.
- Chuẩn hóa tên trường dữ liệu phục vụ quá trình phân tích ngữ nghĩa.

3.1.2 Module xây dựng đồ thị phụ thuộc API (Operation Dependency Graph)

Module `graph_builder.py` có nhiệm vụ xây dựng Đồ thị phụ thuộc hoạt động (Operation Dependency Graph – ODG), biểu diễn mối quan hệ producer–consumer giữa các API endpoint dưới dạng đồ thị có hướng có trọng số.

Inverted Semantic Index. Thay vì so sánh trực tiếp từng cặp API theo kiểu $O(n^2)$, module xây dựng một chỉ mục ngược (Inverted Index) phân nhóm các trường theo loại ngữ nghĩa. Mỗi trường đầu ra và đầu vào được phân loại vào một trong các nhóm: *identity*, *auth/workflow*, *finance* hoặc *unknown*. Chỉ các trường cùng nhóm ngữ nghĩa mới được đưa vào so sánh, giúp giảm đáng kể chi phí tính toán.

Tính điểm tin cậy cạnh. Với mỗi cặp trường (đầu ra của API A, đầu vào của API B) cùng nhóm ngữ nghĩa, hệ thống tính độ tin cậy theo công thức kết hợp:

$$\text{confidence} = \text{base_score} \times w_{\text{stopword}} \times w_{\text{idf}} \times w_{\text{direction}} \quad (3.1)$$

trong đó:

- `base_score`: điểm khớp tên, được tính theo ba lớp theo thứ tự ưu tiên giảm dần – khớp chính xác sau chuẩn hóa (0.95), khớp theo LLM Identity Cluster (0.85), và khớp Jaccard ngữ nghĩa (0.75–0.90).

3.1. Kiến trúc tổng thể của hệ thống

- w_{idf} : hệ số phạt cho trường xuất hiện ở quá nhiều API (IDF Penalty), giúp hạ thấp ưu tiên các trường quá phổ biến như `status`, `type`.
- $w_{stopword}$: hệ số phạt cho trường chỉ chứa một token chung chung như `number`, `state`.
- $w_{direction}$: hệ số điều chỉnh theo chiều phương thức HTTP; cạnh từ GET sang POST bị phạt hệ số 0.6 vì thể hiện chiều phụ thuộc kém tin cậy hơn.

Chỉ các cạnh có $confidence \geq 0.25$ được giữ lại. Các cạnh sau đó được phân hạng theo ba mức: *strong* (≥ 0.75), *medium* (≥ 0.45) và *fallback* (< 0.45).

Safety Net. Sau khi đồ thị được xây dựng, hệ thống quét toàn bộ API để kiểm tra các trường bắt buộc (required) hoặc tham số đường dẫn (path parameter) chưa được cover bởi bất kỳ cạnh nào. Với mỗi trường thiếu, hệ thống tìm provider tốt nhất thông qua fuzzy matching tên trường và tự động inject cạnh dự phòng với $confidence$ từ 0.30 đến 0.40.

Củng cố cạnh theo thời gian chạy (Runtime Edge Reinforcement). Sau khi đồ thị ODG được xây dựng tĩnh, hệ thống tiếp tục điều chỉnh độ tin cậy các cạnh dựa trên kết quả thực thi thực tế. Sau mỗi lần thực thi API, giá trị $confidence$ của cạnh được cập nhật theo quy tắc:

$$confidence' = \begin{cases} \min(1.0, confidence \times 1.1) & \text{nếu request thành công} \\ confidence \times 0.8 & \text{nếu request thất bại (HTTP 400)} \end{cases} \quad (3.2)$$

Nhờ cơ chế này, đồ thị ODG tự tiến hóa trong quá trình Fuzzing: các cạnh phản ánh đúng quan hệ phụ thuộc thực tế được ưu tiên trong các lần duyệt tiếp theo, trong khi các cạnh không hiệu quả bị hạ thấp ưu tiên, giúp chiến lược kiểm thử thích nghi với đặc điểm cụ thể của hệ thống API đang được kiểm thử.

Sau khi thu thập thông tin từ OpenAPI, hệ thống tiến hành xây dựng Đồ thị phụ thuộc hoạt động (Operation Dependency Graph - ODG).

ODG được biểu diễn dưới dạng:

$$G = (V, E) \quad (3.3)$$

Trong đó:

- V là tập hợp các Operation tương ứng với các API Endpoint.
- E là tập hợp các cạnh biểu diễn quan hệ phụ thuộc dữ liệu giữa các Operation.

Mỗi cạnh:

$$e(API_A, API_B) \quad (3.4)$$

thể hiện khả năng dữ liệu được tạo ra bởi API_A có thể được sử dụng làm đầu vào cho API_B .

3.1.3 Module quản lý trạng thái và thực thi (Runtime Executor StateStore)

Runtime Executor chịu trách nhiệm thực thi các chuỗi API dựa trên đồ thị phụ thuộc. Trong quá trình thực thi, StateStore lưu trữ các dữ liệu động bao gồm:

- Authentication Token.
- Session Identifier.
- Resource Identifier.
- Giá trị sinh ra từ Response.

Việc duy trì trạng thái giúp hệ thống có khả năng mô phỏng hành vi của người dùng thực thay vì chỉ gửi các request độc lập.

3.1.4 Module đột biến bất đồng bộ (Async Mutator)

Async Mutator chịu trách nhiệm sinh payload kiểm thử và gửi request đến hệ thống mục tiêu.

Module này bao gồm:

- FuzzDictionary chứa các payload bất thường.
- Payload Mutation Engine.
- AsyncFuzzEngine thực hiện gửi request song song.
- Cơ chế phát hiện lỗi dựa trên HTTP Response.

3.1.5 Module suy luận ngữ nghĩa (Rule Inference Layer)

Module `rule_inference_layer.py` chịu trách nhiệm đánh giá mức độ tương thích về ngữ nghĩa giữa các trường đầu ra của API này và đầu vào của API khác. Quá trình xử lý gồm hai lớp:

- **Lớp dựa trên luật (Rule-based):** Phân loại trường theo các nhóm ngữ nghĩa *identity*, *auth/workflow* và *finance* dựa trên danh sách từ khóa định sẵn.
- **Lớp LLM Fallback:** Với các trường không xác định được, module truy vấn bộ nhớ đệm của LLMPlanner để lấy kết quả phân loại đã được LLM xử lý từ trước.

Độ tin cậy của mỗi cặp trường được tính theo công thức kết hợp:

$$\text{confidence} = \text{base_score} \times w_{\text{stopword}} \times w_{\text{idf}} \times w_{\text{direction}} \quad (3.5)$$

trong đó w_{idf} là hệ số phạt cho trường xuất hiện ở quá nhiều API, w_{stopword} phạt các token quá chung chung, và $w_{\text{direction}}$ phạt theo chiều phương thức HTTP (GET $\rightarrow$ POST được phạt 0.6).

3.1.6 Module sinh payload thông minh (LLM Planner)

Module `llm_planner.py` quản lý giao tiếp với LLM thông qua GitHub Models API. Module cung cấp ba dịch vụ chính:

1. **Phân loại trường (Semantic Classification):** Gửi batch các trường chưa xác định cho LLM để phân loại một lần duy nhất trước khi xây dựng đồ thị.
2. **Gom nhóm thực thể (Identity Clustering):** Nhóm các trường định danh đồng nghĩa (ví dụ: `user_id`, `userId`, `account_id`) vào cùng một cluster để tránh tạo cạnh sai.
3. **Sinh payload (Payload Generation):** Tạo payload JSON hợp lệ dựa trên schema API, trạng thái hiện tại và các phụ thuộc ODG. Khi LLM không khả dụng, module tự động chuyển sang cơ chế heuristic dự phòng.

3.1.7 Module bộ máy chiến lược kiểm thử (Test Strategy Engine)

Module `test_strategy_engine.py` là thành phần trung tâm điều phối toàn bộ quá trình kiểm thử thông qua thuật toán Beam Search trên đồ thị ODG.

Tại mỗi bước duyệt, bộ máy thực hiện:

1. Xác định API tiếp theo trong chuỗi dựa trên các cạnh ODG.
2. Tự động thu thập tham số bắt buộc còn thiếu (resolve missing dependencies) bằng cách gọi đệ quy các API provider.
3. Thực thi request thông qua `RuntimeExecutor`.
4. Sau khi nhận phản hồi thành công (HTTP 2xx), kích hoạt `AsyncFuzzEngine` bắn 50 request đột biến để tìm lỗi 500.
5. Tính điểm heuristic cho beam và cập nhật độ tin cậy cạnh ODG.
6. Chỉ giữ lại W beam có điểm cao nhất (W = beam width) cho bước tiếp theo.

3.1.8 Module thực thi HTTP (Runtime Executor)

Module `runtime_executor.py` chịu trách nhiệm gửi từng request HTTP đồng bộ trong chuỗi kiểm thử có trạng thái. Module xây dựng URL từ path template, đính kèm header xác thực từ `StateStore`, gửi request và phân tích phản hồi để phát hiện lỗi HTTP 500.

3.1.9 Module bộ nhớ thời gian chạy (Knowledge Memory)

Module `knowledge_memory.py` đóng vai trò kho lưu trữ trung tâm trong suốt phiên kiểm thử. Module ghi nhận:

- Lịch sử toàn bộ request (URL, method, status, payload, phản hồi).
- Thống kê trạng thái theo từng endpoint.
- Phản hồi của các cạnh ODG (edge feedback) để hỗ trợ cơ chế củng cố.
- Danh sách lỗ hổng phát hiện được.

Kết thúc phiên kiểm thử, module xuất toàn bộ dữ liệu ra file JSON để `generate_report.py` sinh báo cáo HTML.

3.2 Phân tích tĩnh và xây dựng đồ thị phụ thuộc API

3.2.1 Bài toán xây dựng quan hệ phụ thuộc

Trong hệ thống REST API, nhiều Endpoint có mối quan hệ producer-consumer. Một API có thể tạo ra dữ liệu được sử dụng bởi API khác trong các bước tiếp theo.

Ví dụ:

- API đăng nhập tạo Access Token.
- API tạo tài nguyên tạo Resource ID.
- API cập nhật hoặc xóa sử dụng Resource ID đã tạo.

Do đó, hệ thống cần xác định các quan hệ này để xây dựng chuỗi thực thi hợp lệ.

3.2.2 Inverted Semantic Index

Phương pháp so sánh toàn bộ tham số giữa các API có độ phức tạp lớn.

Với N API và F trường dữ liệu trung bình trên mỗi API, chi phí tìm kiếm quan hệ phụ thuộc là:

$$O(N^2 \times F^2) \quad (3.6)$$

Để tối ưu quá trình này, hệ thống triển khai cấu trúc Inverted Semantic Index.

Các trường dữ liệu được chuẩn hóa và phân nhóm thành các Semantic Bucket:

- Identity Bucket: id, uuid, userId.
- Authentication Bucket: token, accessToken.
- Finance Bucket: amount, price, balance.

Quá trình tìm kiếm quan hệ phụ thuộc chỉ được thực hiện giữa các trường thuộc cùng nhóm ngữ nghĩa, giúp giảm đáng kể không gian tìm kiếm.

3.2.3 Cơ chế tự phục hồi payload (Self-Healing)

Trong quá trình thực thi chuỗi API, payload ban đầu có thể không đáp ứng yêu cầu của server do thiếu trường bắt buộc hoặc sai định dạng. Để tăng khả năng đạt được các API endpoint sâu trong chuỗi phụ thuộc, hệ thống tích hợp cơ chế tự phục hồi (Self-Healing).

Khi nhận được phản hồi lỗi HTTP ≥ 400 , hệ thống thực hiện tối đa 3 vòng lặp sửa chữa: gửi payload lỗi kèm phản hồi của server cho LLM để phân tích, nhận về payload đã hiệu chỉnh, và thực thi lại. Vòng lặp dừng sớm khi nhận được phản hồi thành công.

Để tránh lãng phí tài nguyên, hệ thống áp dụng hai cơ chế giới hạn:

- **Global budget:** Mỗi cặp (API, mã lỗi) chỉ được sửa tối đa 1 lần (với lỗi 500) hoặc 3 lần (với lỗi 4xx) trong toàn bộ phiên.
- **Dedup payload:** Không thực thi lại nếu payload mới trùng với payload đã thử trước đó.

3.2.4 Mô hình tính điểm phụ thuộc

Để quyết định tạo một cạnh trong ODG, hệ thống tính toán độ tin cậy giữa trường đầu ra f_{out} và trường đầu vào f_{in} .

Điểm phụ thuộc được xác định:

$$BaseScore \times W_{penalty} \times D(API_A, API_B)$$

Trong đó:

- $BaseScore$: độ tương đồng ngữ nghĩa.
- $W_{penalty}$: trọng số giảm ảnh hưởng trường phổ biến.
- D : hệ số điều chỉnh theo HTTP Method.

3.2.5 Độ tương đồng Jaccard

Tên trường dữ liệu sau khi chuẩn hóa được tách thành các token.

Độ tương đồng giữa hai tập token được tính bằng:

$$\frac{|T_{out} \cap T_{in}|}{|T_{out} \cup T_{in}|}$$

Điểm cơ sở được xác định:

3.3. Quản lý trạng thái Stateful Execution

$$BaseScore = \min(0.9, 0.75 + J(T_{out}, T_{in}) \times 0.15) \quad (3.7)$$

3.2.6 IDF Penalty Weight

Nhằm giảm các liên kết giả sinh ra bởi các trường phổ biến, hệ thống tính Frequency Ratio:

$$FR(f) = \frac{|API \in N|f \in API|}{|N|} \quad (3.8)$$

Trọng số IDF:

$$W_{IDF}(f) = \begin{cases} 0.25 & FR(f) > 0.30 \\ 0.55 & FR(f) > 0.15 \\ 1.0 & otherwise \end{cases} \quad (3.9)$$

Hệ số phạt tổng:

$$W_{IDF} \times W_{stopword}(f_{out}) \times W_{stopword}(f_{in})$$

3.2.7 Method Directionality Score

Ngoài sự tương đồng dữ liệu, hệ thống xem xét tính hợp lý của quan hệ dựa trên HTTP Method.

$$D(Method_A, Method_B) = \begin{cases} 0.6 & Method_A = GET, Method_B \in POST, PUT, PATCH \\ 1.0 & otherwise \end{cases} \quad (3.10)$$

3.3 Quản lý trạng thái Stateful Execution

3.3.1 Trích xuất trạng thái động

Sau mỗi lần gọi API thành công, Runtime Executor phân tích JSON Response để tìm kiếm các giá trị có khả năng sử dụng trong các bước tiếp theo.

Các giá trị này được lưu vào StateStore và tự động chèn vào các request kế tiếp.

3.3.2 Cơ chế Cloneable StateStore

Khi một chuỗi kiểm thử tạo ra nhiều nhánh thực thi, hệ thống thực hiện sao chép trạng thái hiện tại:

$$State_i = Clone(State_{current}) \quad (3.11)$$

Mỗi nhánh sở hữu một trạng thái độc lập, giúp tránh:

- Race Condition.
- Xung đột dữ liệu giữa các luồng.
- Sai lệch kết quả kiểm thử.

3.4 Phương pháp đột biến và bắn phá bất đồng bộ

3.4.1 Fuzz Dictionary

Hệ thống sử dụng bộ từ điển payload được xây dựng dựa trên mục tiêu phát hiện lỗi. Các nhóm payload bao gồm:

- SQL Injection.
- Cross-site Scripting.
- Null Byte Injection.
- Buffer Overflow String.
- Integer Boundary Values.

3.4.2 Async Payload Blasting

Quy trình thực thi gồm:

1. Nhận request hợp lệ ban đầu.
2. Áp dụng chiến lược Mutation.

3. Sinh tập payload kiểm thử.
4. Gửi request bất đồng bộ.
5. Phân tích Response và ghi nhận lỗi.

Việc sử dụng cơ chế Asynchronous I/O giúp loại bỏ thời gian chờ mạng và tăng số lượng request có thể thực thi trong cùng một khoảng thời gian.

3.5 Runtime Feedback và hệ thống báo cáo

Trong quá trình chạy, hệ thống cập nhật độ tin cậy của các cạnh trong ODG dựa trên kết quả thực tế.

Công thức cập nhật:

$$\min(1.0, C_t(e) \times M_{result})$$

Trong đó:

$$\begin{cases} 1.1 & \text{thực thi thành công 0.8} \\ \text{thất bại} \end{cases}$$

Toàn bộ dữ liệu kiểm thử bao gồm:

- Request.
- Response.
- HTTP Status Code.
- Payload đã sử dụng.
- Attack Chain.

được lưu dưới dạng JSON và chuyển đổi thành Dashboard HTML phục vụ quá trình phân tích.

3.6 Kết luận chương

Chương này đã trình bày phương pháp luận và thiết kế của hệ thống Hybrid Stateful API Fuzzer. Hệ thống kết hợp giữa phân tích tĩnh dựa trên OpenAPI, xây dựng đồ thị phụ thuộc API, quản lý trạng thái Stateful Execution và cơ chế Async Payload Blasting.

Các thuật toán được xây dựng nhằm giúp hệ thống tự động tạo chuỗi kiểm thử hợp lệ, tối ưu quá trình khám phá API và nâng cao khả năng phát hiện các lỗi xử lý bất thường trong RESTful API.

Chương 4

TRIỂN KHAI HỆ THỐNG VÀ THỰC NGHIỆM

Chương này trình bày quá trình hiện thực hóa kiến trúc Hybrid Stateful API Fuzzer được trình bày trong Chương 3. Nội dung bao gồm cấu trúc mã nguồn, cách triển khai các module chính, hiện thực các thuật toán tính toán độ phụ thuộc API, cơ chế quản lý trạng thái và quy trình thực thi kiểm thử.

Ngoài ra, chương này trình bày một kịch bản thực nghiệm nhằm đánh giá khả năng hoạt động của hệ thống trong việc xây dựng chuỗi gọi API, duy trì trạng thái và phát hiện các lỗi xử lý bất thường.

4.1 Môi trường triển khai

Hệ thống được xây dựng dưới dạng một công cụ kiểm thử bảo mật API tự động. Các công nghệ sử dụng trong quá trình phát triển bao gồm:

- **Ngôn ngữ lập trình:** Python 3.x.
- **Thư viện thực thi HTTP:** requests (phiên bản 2.32.3) được sử dụng để gửi request đồng bộ trong chuỗi kiểm thử có trạng thái; aiohttp được sử dụng để gửi request bất đồng bộ trong giai đoạn Robustness Blasting.
- **Framework xử lý bất đồng bộ:** asyncio và aiohttp, được sử dụng để xây dựng cơ chế gửi request đồng thời trong AsyncFuzzEngine.

4.1. Môi trường triển khai

- **Tích hợp mô hình ngôn ngữ lớn:** Thư viện `openai` (phiên bản 1.61.0) kết nối với GitHub Models API, sử dụng mô hình GPT-4o-mini để sinh payload thông minh và phân loại ngữ nghĩa trường dữ liệu.
- **Xác thực dữ liệu:** `pydantic` (phiên bản ≥ 2.0) được sử dụng để xác thực phản hồi JSON từ LLM trước khi đưa vào xử lý.
- **Quản lý cấu hình:** `python-dotenv` hỗ trợ nạp biến môi trường từ file `.env`, bao gồm token xác thực, URL mục tiêu và các biến môi (seed).
- **Thuật toán tìm kiếm:** Beam Search được áp dụng để khám phá chuỗi API theo chiều sâu trong khi kiểm soát sự bùng nổ tổ hợp.
- **Định dạng dữ liệu:** JSON phục vụ việc đọc OpenAPI Specification và lưu trữ kết quả kiểm thử; Graphviz DOT được sử dụng để xuất và trực quan hóa đồ thị phụ thuộc API (ODG).
- **Tài liệu đặc tả API:** OpenAPI Specification (Swagger) được sử dụng làm nguồn dữ liệu đầu vào cho quá trình phân tích.
- **Môi trường kiểm thử:** Các ứng dụng REST API có lỗi hổng được triển khai trong môi trường kiểm soát nhằm đánh giá khả năng phát hiện lỗi.

Hệ thống được phát triển bằng ngôn ngữ Python với mục tiêu tối ưu khả năng xử lý các tác vụ I/O bất đồng bộ.

Các thành phần chính sử dụng trong quá trình triển khai:

- Python: Ngôn ngữ lập trình chính.
- `aiohttp`: Thư viện thực hiện HTTP request bất đồng bộ.
- `asyncio`: Framework quản lý luồng thực thi bất đồng bộ.
- JSON Parser: Xử lý tài liệu OpenAPI và dữ liệu Response.
- Jinja2/HTML Generator: Sinh giao diện báo cáo kết quả.

4.2 Cấu trúc chương trình

Hệ thống được tổ chức theo cấu trúc phẳng (flat structure), trong đó tất cả các module được đặt cùng cấp trong một thư mục duy nhất.

Cấu trúc thư mục chính:

OPG/

```
main.py
spec_parser.py
graph_builder.py
rule_inference_layer.py
llm_planner.py
state_store.py
test_strategy_engine.py
runtime_executor.py
local_mutator.py
knowledge_memory.py
generate_report.py
requirements.txt
```

Trong đó:

- `spec_parser.py`: Phân tích tài liệu OpenAPI và trích xuất thông tin các API endpoint, tham số đầu vào và schema phản hồi.
- `graph_builder.py`: Xây dựng đồ thị phụ thuộc giữa các API dựa trên quan hệ producer-consumer của các trường dữ liệu.
- `rule_inference_layer.py`: Phân loại ngữ nghĩa các trường dữ liệu và tính toán độ tin cậy của quan hệ phụ thuộc giữa hai API.
- `llm_planner.py`: Tích hợp LLM để sinh payload kiểm thử, phân loại trường chưa xác định và tự động sửa payload khi gặp lỗi.
- `state_store.py`: Quản lý trạng thái trong quá trình thực thi, lưu trữ token xác thực và định danh tài nguyên.

- `test_strategy_engine.py`: Bộ máy chiến lược kiểm thử, duyệt đồ thị ODG theo thuật toán Beam Search và điều phối toàn bộ luồng.
- `runtime_executor.py`: Module thực thi HTTP đồng bộ, phân tích phản hồi và thực hiện vòng lặp tự phục hồi payload.
- `local_mutator.py`: Sinh payload đột biến và gửi bất đồng bộ hàng loạt request thông qua `aiohttp`.
- `knowledge_memory.py`: Lưu trữ lịch sử request, thống kê endpoint và tổng hợp kết quả để xuất báo cáo.
- `generate_report.py`: Sinh báo cáo kết quả định dạng HTML.
- `main.py`: Điểm khởi động hệ thống, khởi tạo và kết nối các module theo mô hình Dependency Injection.

4.3 Hiện thực các module chính

4.3.1 Module xây dựng đồ thị phụ thuộc API

Module `graph_builder.py` chịu trách nhiệm triển khai thuật toán xây dựng Operation Dependency Graph.

Thay vì thực hiện so sánh toàn bộ cặp tham số giữa các API, hệ thống sử dụng cấu trúc Inverted Semantic Index để nhóm các trường dữ liệu có cùng ý nghĩa.

Quá trình xử lý gồm:

1. Nhận danh sách API từ Spec Parser.
2. Chuẩn hóa tên trường dữ liệu.
3. Phân loại trường vào Semantic Bucket.
4. So sánh các trường trong cùng nhóm.
5. Tính Confidence Score.
6. Tạo cạnh trong ODG nếu điểm phụ thuộc vượt ngưỡng.

Cách tiếp cận này giúp giảm số lượng phép so sánh cần thực hiện trong quá trình xây dựng đồ thị.

4.3.2 Module quản lý trạng thái StateStore

Module `state_store.py` chịu trách nhiệm lưu trữ các giá trị sinh ra trong quá trình thực thi API.

Trạng thái được tổ chức dưới dạng Key-Value:

```
{  
"access_token": "...",  
"user_id": 123,  
"resource_id": 456  
}
```

Hai chức năng quan trọng:

Trích xuất trạng thái động

Sau mỗi Response thành công, hệ thống phân tích dữ liệu JSON và tìm kiếm các trường có khả năng sử dụng lại như:

- token.
- id.
- uuid.

Nhân bản trạng thái

Khi tạo nhánh kiểm thử mới, StateStore thực hiện sao chép sâu:

$$\text{State}_i = \text{Clone}(\text{State}_{\text{current}}) \quad (4.1)$$

Cơ chế này đảm bảo mỗi chuỗi kiểm thử hoạt động trên một vùng dữ liệu độc lập.

4.3.3 Module Local Mutator và Async Fuzz Engine

Module `local_mutator.py` chịu trách nhiệm sinh dữ liệu kiểm thử và thực hiện quá trình bắn phá payload.

FuzzDictionary

FuzzDictionary lưu trữ các nhóm payload dựa trên kiểu dữ liệu:

- String: SQL Injection, XSS, Path Traversal.
- Integer: Giá trị biên, số âm, số vượt giới hạn.
- Null: None, Null byte.
- Buffer: Chuỗi có kích thước lớn.

Ví dụ với kiểu dữ liệu Integer:

```
[  
-2147483648,  
0,  
9999999999,  
"abc",  
None  
]
```

AsyncFuzzEngine

AsyncFuzzEngine sử dụng thư viện aiohttp kết hợp với asyncio để gửi nhiều request đồng thời.

Luồng xử lý:

1. Nhận Endpoint cần kiểm thử.
2. Sinh danh sách payload.
3. Tạo các task bất đồng bộ.
4. Gửi request song song.
5. Thu thập HTTP Response.
6. Phân tích lỗi.

4.4 Hiện thực các thuật toán toán học

4.4.1 Tính Jaccard Similarity

Công thức Jaccard trong Chương 3 được hiện thực bằng cấu trúc dữ liệu Set của Python.

```
intersection = set1.intersection(set2)
```

```
union = set1.union(set2)
```

```
score = len(intersection)/len(union)
```

Việc sử dụng Set giúp giảm thời gian xử lý khi so sánh các tập token.

4.4.2 Tính IDF Penalty Weight

Hệ thống xây dựng bộ nhớ đệm lưu trữ tần suất xuất hiện của từng trường dữ liệu. Trọng số được tính theo:

```
if freq > 0.30:
```

```
weight = 0.25
```

```
elif freq > 0.15:
```

```
weight = 0.55
```

```
else:
```

```
weight = 1.0
```

4.4.3 Tính Confidence Score

Điểm tin cậy cuối cùng được tổng hợp từ:

- Similarity Score.
- IDF Weight.
- Stopword Weight.

- Method Directionality.

Kết quả này được sử dụng để quyết định việc tạo cạnh giữa hai API trong ODG.

4.5 Quy trình thực thi hệ thống

Quy trình hoạt động tổng quát:

1. Người dùng cung cấp file OpenAPI.
2. Spec Parser phân tích tài liệu.
3. Graph Builder tạo ODG.
4. Runtime Executor duyệt đồ thị.
5. StateStore duy trì dữ liệu thực thi.
6. Local Mutator sinh payload.
7. AsyncFuzzEngine gửi request kiểm thử.
8. Generate Report tạo báo cáo kết quả.

4.6 Kịch bản thực nghiệm

Hệ thống được kiểm thử trên một tập API RESTful có các Endpoint yêu cầu xác thực.

4.6.1 Giai đoạn xây dựng ngữ cảnh

Đầu tiên, hệ thống thực hiện gọi API đăng nhập.

Response trả về chứa Access Token.

StateStore tự động phát hiện và lưu trữ giá trị này để sử dụng trong các request tiếp theo.

4.6.2 Giai đoạn Fuzzing

Sau khi có trạng thái hợp lệ, hệ thống tiếp tục kiểm thử các API phụ thuộc. Các tham số đầu vào được thay thế bằng các payload trong FuzzDictionary.

Ví dụ:

- Giá trị âm.
- Giá trị vượt giới hạn.
- Chuỗi ký tự đặc biệt.
- Dữ liệu rỗng.

Các request được gửi đồng thời thông qua AsyncFuzzEngine.

4.6.3 Giai đoạn ghi nhận kết quả

Sau khi hoàn thành quá trình kiểm thử, hệ thống lưu lại:

- Endpoint.
- HTTP Method.
- Payload.
- Request Header.
- Response Code.
- Response Body.

Các dữ liệu này được chuyển đổi thành Dashboard HTML nhằm hỗ trợ việc phân tích và tái hiện lỗi.

4.7 Cơ chế tự phục hồi payload (Self-Healing)

Trong quá trình thực thi kiểm thử, một request hợp lệ có thể thất bại do payload không đáp ứng đúng yêu cầu của server, ví dụ như trường thiếu dữ liệu bắt buộc hoặc

giá trị không hợp lệ. Để xử lý vấn đề này, module `runtime_executor.py` tích hợp cơ chế tự phục hồi (Self-Healing) dựa trên phản hồi thực tế của server.

Quy trình tự phục hồi hoạt động như sau:

1. Thực thi request với payload ban đầu do LLM hoặc Heuristic sinh ra.
2. Nếu phản hồi trả về mã lỗi HTTP ≥ 400 , hệ thống gửi payload lỗi và nội dung phản hồi của server cho LLM để phân tích và sửa.
3. LLM sinh ra payload đã được hiệu chỉnh dựa trên thông tin lỗi.
4. Hệ thống thực thi lại với payload mới.
5. Vòng lặp được lặp tối đa 3 lần hoặc cho đến khi nhận được phản hồi thành công.

Cơ chế này giúp giảm số lượng request thất bại không cần thiết và tăng khả năng đạt được các API endpoint sâu hơn trong chuỗi phụ thuộc, từ đó mở rộng vùng kiểm thử của Robustness Fuzzer.

4.8 Kết luận chương

Chương này đã trình bày quá trình triển khai hệ thống Hybrid Stateful API Fuzzer từ kiến trúc thiết kế đến hiện thực mã nguồn.

Các thuật toán xây dựng đồ thị phụ thuộc, quản lý trạng thái và cơ chế Async Payload Blasting được hiện thực thành các module độc lập. Kết quả thực nghiệm cho thấy hệ thống có khả năng tự động xây dựng chuỗi thực thi API, duy trì trạng thái và phát hiện các phản hồi bất thường trong quá trình kiểm thử.

Chương 5

THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.0.1 Thiết lập môi trường và Tiêu chuẩn đánh giá

Quá trình đánh giá định lượng được thực hiện trên hai hệ thống mã nguồn mở được thiết kế chuyên biệt và sử dụng rộng rãi làm tiêu chuẩn tham chiếu trong các nghiên cứu về bảo mật API:

1. **VAmPI (Vulnerable API):** Ứng dụng được xây dựng trên nền tảng Python/Flask kết hợp cơ sở dữ liệu SQLite, cố tình cài cắm các lỗ hổng bám sát danh mục rủi ro OWASP API Security Top 10. Hệ thống này mô phỏng chân thực các bề mặt tấn công phổ biến như Broken Object Level Authorization (BOLA), SQL Injection và Mass Assignment, cho phép đánh giá chính xác độ nhạy của các công cụ kiểm thử.
2. **crAPI (Completely Ridiculous API):** Dự án mã nguồn mở do chính tổ chức OWASP phát triển, mô phỏng một ứng dụng B2C (nền tảng dịch vụ bảo trì xe ô tô) với kiến trúc vi dịch vụ (Microservices) phức tạp. Backend của crAPI được triển khai phân tán với nhiều công nghệ thực tế bao gồm Java (Identity service), Python (Workshop service), Go (Community service) và Node.js. Các lỗ hổng logic nghiệp vụ và phân quyền trong crAPI được ẩn giấu tinh vi, đòi hỏi Fuzzer phải có khả năng khám phá và khai thác theo chuỗi trạng thái nhiều bước (multi-step exploitation).

Toàn bộ các công cụ kiểm thử tham gia đối chứng, bao gồm hệ thống đề xuất **Hybrid Stateful API Fuzzer**, **RESTler** và **EvoMaster**, đều được vận hành theo cơ chế hộp đen (Black-box) thuần túy. Lưu lượng HTTP giao tiếp giữa các công cụ và hệ thống mục tiêu được định tuyến và giám sát chặt chẽ thông qua một máy chủ proxy trung gian.

Nhằm đảm bảo tính khách quan của dữ liệu và loại bỏ yếu tố sai số ngẫu nhiên do độ trễ mạng, mỗi cặp (công cụ, môi trường) được chạy thử nghiệm lặp lại $N = 5$ lần với ngân sách thời gian thực thi cố định là 1,800 giây (30 phút). Toàn bộ quá trình kiểm thử được triển khai cô lập trên các ngăn xếp Docker (isolated Docker stacks) để ngăn chặn hiện tượng nhiễu tài nguyên. Các lỗi hỏng phát hiện sau cùng được hệ thống tự động lọc trùng lặp và định nghĩa chuẩn hóa theo cặp (thao tác API, tệp nguồn: dòng) nhằm phục vụ quá trình đối chiếu hiệu năng.

5.0.2 Báo cáo kết quả định lượng tổng hợp

Các số liệu thu được về tổng số request thực tế gửi đi ($|R|$), số lỗi độc nhất phát hiện, và mật độ lỗi tương quan với kích thước mã nguồn được tổng hợp chi tiết trong Bảng 5.1:

Bảng 5.1: Mức độ bao phủ và mật độ lỗi hỏng phần mềm phía sau cho mỗi công cụ

Môi trường	Công cụ đối chứng	Endpoint tương tác	Requests ($ R $)	Lỗi Bugs	$BPR_{\text{HTTP 500}}$
VAmPI	RESTler	??	??	??	??
	EvoMaster	21	2331	18	7
	Hybrid Stateful API Fuzzer	7	??	7	5
crAPI	RESTler	??	??	??	??
	EvoMaster	21	137	45.0	5.0
	Hybrid Stateful API Fuzzer	34	201	14	12

Chương 6

ĐỀ CƯƠNG KHÓA LUẬN TỐT NGHIỆP

6.1 Đặt vấn đề và Tổng quan

Đồ án chuyên ngành đã xây dựng thành công kiến trúc cốt lõi của hệ thống Hybrid Stateful API Fuzzer. Kết quả đạt được cho thấy việc kết hợp phương pháp phân tích tĩnh (Static Analysis) thông qua thuật toán Inverted Semantic Index và cơ chế quản lý trạng thái cô lập (StateStore) có khả năng tự động hóa quá trình xây dựng chuỗi gọi API, đồng thời hỗ trợ phát hiện các lỗi xử lý bất thường như lỗi máy chủ HTTP 500 với hiệu suất cao.

Tuy nhiên, phương pháp hiện tại chủ yếu dựa trên các luật suy luận tĩnh (Heuristic Rules) và kỹ thuật đột biến dữ liệu dựa trên từ điển (Payload Blasting). Cách tiếp cận này có khả năng phát hiện các lỗi biên (Boundary Errors) hoặc lỗi xử lý dữ liệu bất thường, nhưng vẫn còn hạn chế trong việc phát hiện các lỗ hổng liên quan đến logic nghiệp vụ phức tạp.

Các dạng lỗ hổng như BOLA/IDOR (Broken Object Level Authorization), BFLA (Broken Function Level Authorization) thường không gây ra lỗi hệ thống (HTTP 500), mà biểu hiện thông qua việc người dùng có thể truy cập hoặc thao tác trái phép tài nguyên của người dùng khác mặc dù hệ thống vẫn trả về phản hồi hợp lệ (HTTP 200).

Do đó, khóa luận tốt nghiệp hướng tới việc mở rộng hệ thống hiện tại bằng cách tích hợp kiến trúc Đa tác tử Trí tuệ nhân tạo (Multi-Agent LLM), nhằm mô phỏng khả năng

suy luận của chuyên gia bảo mật trong việc lập kế hoạch tấn công, phân tích ngữ nghĩa API và tự động phát hiện các lỗ hổng logic nghiệp vụ trong RESTful API.

Câu hỏi nghiên cứu chính của khóa luận được đặt ra như sau:

"Làm thế nào để tích hợp hệ thống Multi-Agent LLM vào nền tảng Stateful API Fuzzer nhằm tự động xây dựng chiến lược kiểm thử, suy luận chuỗi tấn công và phát hiện các lỗ hổng logic nghiệp vụ phức tạp trong RESTful API?"

6.2 Mục tiêu và Nhiệm vụ nghiên cứu

6.2.1 Mục tiêu

Mục tiêu chính của khóa luận là xây dựng và đánh giá một hệ thống kiểm thử bảo mật tự trị (Autonomous Security Testing) dựa trên kiến trúc Multi-Agent LLM. Hệ thống có khả năng tự động phân tích cấu trúc API, suy luận ngữ nghĩa, xây dựng chuỗi khai thác và đánh giá phản hồi nhằm phát hiện các lỗ hổng bảo mật mà các phương pháp fuzzing truyền thống khó tiếp cận.

6.2.2 Nhiệm vụ cụ thể

Các nhiệm vụ chính của khóa luận bao gồm:

- Nghiên cứu các mô hình ngôn ngữ lớn (Large Language Model - LLM) và các kiến trúc Multi-Agent như LangGraph, AutoGen trong lĩnh vực kiểm thử bảo mật tự động.
- Thiết kế và triển khai kiến trúc Agent Squad với các tác tử chuyên biệt, bao gồm Planner Agent, Executor Agent, Mutator Agent và Evaluator Agent.
- Nâng cấp cơ chế quản lý trạng thái StateStore thành bộ nhớ ngữ cảnh (Knowledge Memory), cho phép hệ thống lưu trữ lịch sử thực thi và tái sử dụng thông tin trong quá trình kiểm thử.
- Phát triển cơ chế tìm kiếm chuỗi tấn công thông minh dựa trên đồ thị phụ thuộc API (Operation Dependency Graph - ODG), kết hợp các thuật toán tìm kiếm như Beam Search, BFS hoặc MCTS nhằm tối ưu không gian trạng thái.

- Đánh giá hiệu quả của hệ thống thông qua việc so sánh với các công cụ kiểm thử API hiện có như RESTler, OWASP ZAP và Schemathesis.

6.3 Phương pháp luận và Nội dung dự định thực hiện

Khóa luận tập trung phát triển hai hướng nghiên cứu chính: xây dựng cơ chế lập kế hoạch tấn công tự động dựa trên Multi-Agent LLM và tối ưu hóa quá trình khám phá chuỗi API thông qua các thuật toán tìm kiếm thông minh.

6.3.1 Phát triển phương pháp lập kế hoạch tấn công tự động bằng Multi-Agent LLM

Thay vì sử dụng một mô hình AI duy nhất để thực hiện toàn bộ quá trình kiểm thử, hệ thống được thiết kế theo mô hình nhiều tác tử chuyên biệt, trong đó mỗi Agent đảm nhận một nhiệm vụ riêng.

Planner Agent

Planner Agent chịu trách nhiệm phân tích cấu trúc API, đồ thị phụ thuộc và mục tiêu kiểm thử để xây dựng kế hoạch khai thác.

Ví dụ, với mục tiêu kiểm tra lỗi BOLA, Agent có thể xây dựng chuỗi thao tác:

- Đăng nhập bằng tài khoản người dùng A.
- Thu thập định danh tài nguyên của người dùng A.
- Chuyển sang tài khoản người dùng B.
- Thay đổi định danh tài nguyên và kiểm tra khả năng truy cập trái phép.

Executor Agent

Executor Agent nhận kế hoạch từ Planner Agent, quản lý trạng thái thực thi, gán các giá trị động như Token, Session ID, Resource ID và gửi request đến hệ thống mục tiêu.

Evaluator Agent

Evaluator Agent chịu trách nhiệm phân tích phản hồi từ hệ thống, đánh giá các dấu hiệu bất thường như:

- Rò rỉ dữ liệu trái phép.
- Sai lệch quyền truy cập.
- Bypass cơ chế xác thực.
- Thay đổi dữ liệu ngoài phạm vi cho phép.

Kết quả phân tích được phản hồi lại cho Planner Agent nhằm điều chỉnh chiến lược kiểm thử.

6.3.2 Tối ưu hóa quá trình tìm kiếm chuỗi tấn công

Do không gian trạng thái của hệ thống API có thể mở rộng rất lớn, khóa luận nghiên cứu áp dụng các thuật toán tìm kiếm thông minh nhằm giảm chi phí khám phá.

Hybrid Graph Search

Hệ thống kết hợp BFS và Beam Search trong quá trình duyệt đồ thị phụ thuộc API.

BFS đảm nhiệm việc mở rộng không gian tìm kiếm theo từng mức độ sâu, trong khi Beam Search thực hiện lựa chọn các nhánh có khả năng dẫn tới lỗ hổng cao dựa trên hàm đánh giá:

$$Score = \alpha C_{dependency} + \beta C_{security} + \gamma C_{historical}$$

Trong đó:

- $C_{dependency}$: độ tin cậy của quan hệ phụ thuộc API.
- $C_{security}$: mức độ nghi ngờ tồn tại lỗ hổng.
- $C_{historical}$: thông tin từ các lần kiểm thử trước.

Knowledge Memory và Vector Database

Các kết quả phân tích API, payload và lịch sử khai thác được chuyển đổi thành vector embedding và lưu trữ trong Vector Database.

Khi gặp một API có cấu trúc tương đồng, hệ thống có thể truy vấn lại thông tin trước đó để tái sử dụng chiến lược kiểm thử, giúp giảm thời gian tìm kiếm.

6.4 Đánh giá kết quả

6.4.1 Dữ liệu thực nghiệm

Hệ thống được đánh giá trên các bộ dữ liệu API có chủ đích chứa lỗ hổng bảo mật, bao gồm:

- crAPI (Completely Ridiculous API).
- vAPI.
- Các hệ thống API thực tế được xây dựng trong môi trường kiểm thử.

6.4.2 Mô hình đối chứng

Kết quả được so sánh với các công cụ kiểm thử bảo mật API phổ biến:

- RESTler.
- OWASP ZAP Active Scanner.
- Schemathesis.

6.4.3 Các tiêu chí đánh giá

Các chỉ số đánh giá bao gồm:

- Tỷ lệ phát hiện lỗ hổng thật (True Positive Rate).
- Tỷ lệ cảnh báo sai (False Positive Rate).
- Thời gian phát hiện chuỗi khai thác đầu tiên (Time-to-Exploit).

- Độ bao phủ API (API Path Coverage).
- Chi phí thực thi (Execution Cost).

6.4.4 Tiêu chí thành công

Hệ thống được kỳ vọng có khả năng phát hiện các lỗi hỏng logic nghiệp vụ yêu cầu nhiều bước thực thi liên tiếp, ví dụ các lỗi BOLA/BFLA cần kết hợp nhiều API và trạng thái người dùng.

So với các công cụ fuzzing truyền thống, hệ thống Multi-Agent có khả năng tận dụng thông tin ngữ nghĩa, trạng thái thực thi và khả năng suy luận của LLM để khám phá các chuỗi tấn công phức tạp mà các phương pháp dựa trên luật cố định khó phát hiện.

Phụ lục A

Hướng dẫn soạn phụ lục

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Tài liệu tham khảo

- [1] Apiary. *Dredd: HTTP API Testing Framework*. <https://github.com/apiaryio/dredd>. 2024.
- [2] Andrea Arcuri. “RESTful API Automated Test Case Generation with EvoMaster”. In: *ACM Transactions on Software Engineering and Methodology* 28.1 (2019), pp. 1–37. DOI: [10.1145/3293455](https://doi.org/10.1145/3293455).
- [3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. “RESTler: Stateful REST API Fuzzing”. In: *IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 748–758. DOI: [10.1109/ICSE.2019.00083](https://doi.org/10.1109/ICSE.2019.00083).
- [4] OWASP Foundation. *OWASP ZAP*. <https://www.zaproxy.org>. 2024.
- [5] PortSwigger. *Burp Suite Documentation*. <https://portswigger.net/burp/documentation>. 2024.
- [6] Schemathesis. *Schemathesis: Property-Based Testing for OpenAPI*. <https://schemathesis.readthedocs.io>. 2024.